

Spring AI RAG（检索增强生成）学习笔记

来源：Spring AI 2.0.1 参考文档

<https://docs.springframework.org.cn/spring-ai/reference/api/retrieval-augmented-generation.html>

一、概述

RAG (Retrieval-Augmented Generation, 检索增强生成) 用于解决大模型在以下方面的短板：

- 长文本处理能力有限
- 事实准确性不足
- 缺乏上下文感知

Spring AI 提供模块化架构：既可以自行搭建 RAG 流程，也可以通过 Advisor API 直接使用开箱即用的方案。

二、Advisors（开箱即用方案）

需要添加 `spring-ai-vector-store-advisor` 依赖。

1. QuestionAnswerAdvisor

- 把用户问题拿去向量库做相似度检索，将结果拼进提示词。
- 通过 `SearchRequest` 设置 `similarityThreshold`（相似度阈值）、`topK`（返回条数）。
- 支持 `FILTER_EXPRESSION` 在运行时做动态过滤。
- 可用 `.promptTemplate()` 自定义提示词模板，模板中必须包含两个占位符：
 - `query`
 - `question_answer_context`

2. RetrievalAugmentationAdvisor

需要添加 `spring-ai-rag` 依赖。

- 基于模块化架构，覆盖从朴素 RAG 到高级 RAG 的各种场景。
- 默认不允许空上下文（上下文为空时让模型拒答）；可用 `allowEmptyContext(true)` 放开限制。
- 高级用法：可加入 `RewriteQueryTransformer` 等查询转换器，以及 `DocumentPostProcessor` 做重排 / 压缩。

三、模块化 RAG（灵感来自论文《Modular RAG》）

1. 检索前（Pre-Retrieval）

- 查询转换：Compression（压缩）、Rewrite（重写）、Translation（翻译）
- 查询扩展：MultiQueryExpander（多查询扩展），建议使用低温度 0.0

2. 检索（Retrieval）

- VectorStoreDocumentRetriever: 支持元数据过滤、相似度阈值、topK。
 - 过滤表达式可为: 静态 / 动态 / 按请求
- 文档合并: ConcatenationDocumentJoiner

3. 检索后 (Post-Retrieval)

- 文档后处理, 用于解决:
 - “迷失在中间” (Lost in the Middle) 问题
 - 上下文长度限制
 - 上下文降噪

4. 生成 (Generation)

- 查询增强: ContextualQueryAugmenter